Федеральное государственное автономное образовательное учреждение высшего образования
Национальный исследовательский университет ИТМО

Факультет Программной Инженерии и Компьютерной Техники

Лабораторная работа №1

по дисциплине

«Распределённые системы хранения данных»:

Вариант 706
Выполнили:
Ястребов-Амирханов Алекси
Исупов Никита
Группа: P3332/P3331
Преподаватель: Афанасьев Дмитрий Борисович

Санкт-Петербург,

2026

Оглавление

Задание....3
Этап 1. Инициализация кластера БД....4
Этап 2. Конфигурация и запуск сервера БД....5
Этап 3. Дополнительные табличные пространства и наполнение базы....8
Демонстрация....10
Вывод....13

Задание.

Цель работы – на выделенном узле создать и сконфигурировать новый кластер БД Postgres, саму БД, табличные пространства и новую роль, а также произвести наполнение базы в соответствии с заданием. Отчёт по работе должен содержать все команды по настройке, скрипты, а также измененные строки конфигурационных файлов.

Этап 1. Инициализация кластера БД

1. Директория кластера: $HOME/wml19
2. Кодировка: KOI8-R
3. Локаль: русская
4. Параметры инициализации задать через аргументы команды

Этап 2. Конфигурация и запуск сервера БД

1. Способы подключения: 1) Unix-domain сокет в режиме peer; 2) сокет TCP/IP, принимать подключения к любому IP-адресу узла
2. Номер порта: 9706
3. Способ аутентификации TCP/IP клиентов: по паролю SHA-256
4. Остальные способы подключений запретить.
5. Настроить следующие параметры сервера БД:
    1. max_connections
    2. shared_buffers
    3. temp_buffers
    4. work_mem
    5. checkpoint_timeout
    6. effective_cache_size
    7. fsync
    8. commit_delay
6. Параметры должны быть подобраны в соответствии с аппаратной конфигурацией: оперативная память 32ГБ, хранение на твердотельном накопителе (SSD).
7. Директория WAL файлов: $PGDATA/pg_wal
8. Формат лог-файлов: .log
9. Уровень сообщений лога: ERROR
10. Дополнительно логировать: завершение сессий и продолжительность выполнения команд

Этап 3. Дополнительные табличные пространства и наполнение базы

1. Создать новые табличные пространства для партицированной таблицы: $HOME/ahq84, $HOME/kbs76
2. На основе template0 создать новую базу: uglyorangebird
3. Создать новую роль, предоставить необходимые права, разрешить подключение к базе.
4. От имени новой роли (не администратора) произвести наполнение ВСЕХ созданных баз тестовыми наборами данных. ВСЕ табличные пространства должны использоваться по назначению.
5. Вывести список всех табличных пространств кластера и содержащиеся в них объекты.

Этап 1. Инициализация кластера БД.

Сначала создадим директорию для кластера:

```
mkdir -p $HOME/wml19
```

Инициализируем кластер PostgreSQL с указанными параметрами:

```
initdb --encoding=KOI8-R --locale=ru_RU.KOI8-R \
  --lc-messages=ru_RU.KOI8-R --lc-monetary=ru_RU.KOI8-R \
  --lc-numeric=ru_RU.KOI8-R --lc-time=ru_RU.KOI8-R \
  -D $HOME/wml19
```

Директория WAL файлов по умолчанию создается как

$HOME/wml19/pg_wal

Где $HOME/wml19 — директория кластера.

Этап 2. Конфигурация и запуск сервера БД.

Расчет параметров сервера БД:

- max_connections = 200

Количество одновременных подключений по умолчанию – 100. Для пиковых нагрузок берем с запасом, исключая ошибки «too many connections»

- shared_buffers = 512 MB

Ограничение виртуальной машины на helios в 512MB не дает сделать больше (хотя оптимальным было бы взять 25% памяти от всего ОЗУ, то есть 8GB)

- temp_buffers = 8 MB

Нет никакой информации про временные объекты поэтому было принято решение оставить значение по умолчанию

- work_mem = 16 Mb

RAM * 0.25 / max_connections = 32GB * 0.25 / 200 = 40.96MB учитывая что мы не знаем сложность возможных операций лучше под завязку не брать. Так же согласно документации PostgreSQL это значение лучше брать, как степень двойки.

- checkpoint_timeout = 5 min

Допустимый диапазон составляет от 30 секунд до одного дня. Значение по умолчанию - пять минут, его и решили оставить

- effective_cache_size = 24 GB

Указывает планировщику, что 75% RAM (32GB – 8GB) доступно для кэша ОС.

Помогает выбирать оптимальные планы запросов (например, индексные сканы вместо seqscan), так как реальные данные часто кэшируются ОС.

- fsync = on

Гарантирует, что данные физически записаны на диск перед подтверждением транзакции. Несмотря на снижение TPS на 10-15%, исключает потерю данных при аварийном отключении.

- commit_delay = 1000

Значение 1000 находится в диапазоне рекомендуемых значений (200–1200). При высокой нагрузке системы задержка позволяет сгруппировать больше транзакций и выполнить их фиксацию через одну операцию записи в WAL, что может повысить пропускную способность.

Настройка параметров подключения:

Отредактируем файл $HOME/jno88/postgresql.conf

```
cat > $HOME/wml19/postgresql.conf << EOF

listen_addresses = '*'
port = 9706
unix_socket_directories = '$HOME/wml19'
password_encryption = 'scram-sha-256'

max_connections = 200
shared_buffers = 512MB
temp_buffers = 8MB
work_mem = 16MB
checkpoint_timeout = 5min
effective_cache_size = 24GB
fsync = on
commit_delay = 1000

log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_min_messages = ERROR
log_statement = 'none'
log_duration = on
log_connections = on
log_disconnections = on
EOF
```

Методы аутентификации определяются в файле pg_hba.conf

```
cat > $HOME/wml19/pg_hba.conf << EOF
local	all	all	peer
host	all	all	0.0.0.0/0	scram-sha-256
EOF
```

local all all peer - Unix domain socket соединения (peer аутентификация)
host all all 0.0.0.0/0 scram-sha-256 - Подключения по IPv4 с любого адреса (аутентификация по паролю SHA-256)

Запуск сервер PostgreSQL:

```
pg_ctl -D $HOME/wml19 -l $HOME/wml19/server.log start
```

Способы подключения:

```
# Подключение через Unix-сокет
psql -h /var/db/postgres0/wml19 -p 9706 postgres

# Подключение через TCP/IP (IPv4)
psql -h 127.0.0.1 -p 9706 postgres
```

Для того чтобы sha-256 работал, надо задать пароль для postgres0 пользователя:

```
# Подключаемся через Unix-сокет (не требует пароля)
psql -h /var/db/postgres0/wml19 -p 9706 postgres

# Внутри psql устанавливаем пароль для пользователя postgres
ALTER USER postgres0 PASSWORD '1234';
```

Этап 3. Дополнительные табличные пространства и наполнение базы

Создание табличных пространств для партицированной таблицы:

```
mkdir -p $HOME/ahq84
mkdir -p $HOME/kbs76

# Создание табличных пространств
psql -h pg197 -p 9706 -U postgres0 postgres -c "CREATE TABLESPACE ts0 LOCATION '$HOME/kbs76';"
psql -h pg197 -p 9706 -U postgres0 postgres -c "CREATE TABLESPACE ts1 LOCATION '$HOME/ahq84';"

# Создание новой базы данных
psql -h pg197 -p 9706 -U postgres0 postgres -c "CREATE DATABASE uglyorangebird TEMPLATE template0;"
```

Создаем новую роль, предоставляем необходимые права, разрешаем подключение к БД:

```
psql -h pg197 -p 9706 -U postgres0 postgres -c "CREATE ROLE test_user
WITH LOGIN PASSWORD 'test123';"

psql -h pg197 -p 9706 -U postgres0 postgres -c "GRANT CONNECT ON
DATABASE uglyorangebird TO test_user;"

psql -h pg197 -p 9706 -U postgres0 postgres -c "GRANT CONNECT ON
DATABASE template0 TO test_user;"

psql -h pg197 -p 9706 -U postgres0 uglyorangebird -c "GRANT ALL ON
SCHEMA public TO test_user;"

psql -h pg197 -p 9706 -U postgres0 postgres -c "GRANT CREATE ON
TABLESPACE ts0 TO test_user;"

psql -h pg197 -p 9706 -U postgres0 postgres -c "GRANT CREATE ON
TABLESPACE ts1 TO test_user;"
```

Создание тестовых таблиц

```
CREATE TABLE event_logs (
    id          bigserial,
    user_id     int NOT NULL,
    event_time  timestamp NOT NULL,
    event_type  text NOT NULL,
    payload     jsonb
) PARTITION BY RANGE (event_time);

CREATE TABLE event_logs_2025
PARTITION OF event_logs
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01')
TABLESPACE ts1;
```

```
CREATE TABLE event_logs_2026
PARTITION OF event_logs
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01')
TABLESPACE ts2;

INSERT INTO event_logs (user_id, event_time, event_type, payload)
SELECT
   (random() * 1000)::int,
   timestamp '2025-01-01'
      + (random() * interval '730 days'),
   (ARRAY['login','logout'])[floor(random()*2)+1],
   jsonb_build_object('ip', md5(random()::text))
FROM generate_series(1,1000);
```

Проверка списков табличных пространств

```
SELECT spcname, pg_tablespace_location(oid) FROM pg_tablespace;
```

Проверка объектов в табличных пространствах

```
SELECT t.spcname, c.relname
FROM pg_class c
JOIN pg_tablespace t ON c.reltablespace = t.oid
WHERE c.relkind = 'r';
```

Скрипт очистки

```
rm -rf $HOME/ahq84
rm -rf $HOME/kbs76
rm -rf $HOME/wml19
```

Демонстрация

Инициализируем кластер:

```
[postgres0@pg197 ~]$ mkdir -p $HOME/wml19
[postgres0@pg197 ~]$ ls
wml19
[postgres0@pg197 ~]$ initdb --encoding=KOI8-R --locale=ru_RU.KOI8-R \
  --lc-messages=ru_RU.KOI8-R --lc-monetary=ru_RU.KOI8-R \
  --lc-numeric=ru_RU.KOI8-R --lc-time=ru_RU.KOI8-R \
  -D $HOME/wml19
Файлы, относящиеся к этой СУБД, будут принадлежать пользователю "postgres0".
От его имени также будет запускаться процесс сервера.

Кластер баз данных будет инициализирован с локалью "ru_RU.KOI8-R".
Выбрана конфигурация текстового поиска по умолчанию "russian".

Контроль целостности страниц данных отключён.

исправление прав для существующего каталога /var/db/postgres0/wml19... ок
создание подкаталогов... ок
выбирается реализация динамической разделяемой памяти... posix
выбирается значение max_connections по умолчанию... 100
выбирается значение shared_buffers по умолчанию... 128MB
выбирается часовой пояс по умолчанию... Europe/Moscow
создание конфигурационных файлов... ок
выполняется подготовительный скрипт... ок
выполняется заключительная инициализация... ок
сохранение данных на диске... ок

initdb: предупреждение: включение метода аутентификации "trust" для локальных подключений
initdb: подсказка: Другой метод можно выбрать, отредактировав pg_hba.conf или ещё раз запустив initdb с ключом -A, --auth-local или --auth-host.

Готово. Теперь вы можете запустить сервер баз данных:

    pg_ctl -D /var/db/postgres0/wml19 -l файл_журнала start

[postgres0@pg197 ~]$ ls
wml19
[postgres0@pg197 ~]$ ls wml19/
base                pg_hba.conf         pg_notify           pg_stat             pg_twophase         postgresql.auto.conf
global              pg_ident.conf       pg_replslot         pg_stat_tmp         PG_VERSION          postgresql.conf
pg_commit_ts        pg_logical          pg_serial           pg_subtrans         pg_wal
pg_dynshmem         pg_multixact        pg_snapshots        pg_tblspc           pg_xact
[postgres0@pg197 ~]$
```

Подключение:

```
[postgres0@pg197 ~]$ psql -h /var/db/postgres0/wml19 -p 9706 postgres
psql (16.4)
Введите "help", чтобы получить справку.

postgres=# ALTER USER postgres0 PASSWORD '1234';
ALTER ROLE
```

```
[postgres0@pg197 ~]$ psql -h pg197 -p 9706 -U postgres0 postgres
Пароль пользователя postgres0:
psql (16.4)
Введите "help", чтобы получить справку.

postgres=#
```

Создание тестовых данных:

```
[postgres0@pg197 ~]$ psql -U test_user -d uglyorangebird -h 127.0.0.1 -p 9706
Пароль пользователя test_user:
psql (16.4)
Введите "help", чтобы получить справку.

uglyorangebird=> CREATE TABLE event_logs (
    id          bigserial,
    user_id     int NOT NULL,
    event_time  timestamp NOT NULL,
    event_type  text NOT NULL,
    payload     jsonb
) PARTITION BY RANGE (event_time);
CREATE TABLE
uglyorangebird=> CREATE TABLE event_logs_2025
PARTITION OF event_logs
FOR VALUES FROM ('2025-01-01') TO ('2026-01-01')
TABLESPACE ts1;
```

```
uglyorangebird=> CREATE TABLE event_logs_2026
PARTITION OF event_logs
FOR VALUES FROM ('2026-01-01') TO ('2027-01-01')
TABLESPACE ts0;
CREATE TABLE
uglyorangebird=> INSERT INTO event_logs (user_id, event_time, event_type, payload)
SELECT
    (random() * 1000)::int,
    timestamp '2025-01-01'
        + (random() * interval '730 days'),
    (ARRAY['login','logout'])[floor(random()*2)+1],
    jsonb_build_object('ip', md5(random()::text))
FROM generate_series(1,1000);
INSERT 0 1000
```

```
uglyorangebird=> SELECT
    tableoid::regclass AS partition,
    count(*)
FROM event_logs
GROUP BY 1
ORDER BY 1;
    partition     | count
------------------+-------
 event_logs_2025 |   454
 event_logs_2026 |   546
(2 строки)
```

Проверка объектов в табличных пространствах:

```
uglyorangebird=> SELECT spcname, pg_tablespace_location(oid) FROM pg_tablespace;
  spcname   | pg_tablespace_location
------------+------------------------
 pg_default |
 pg_global  |
 ts0        | /var/db/postgres0/kbs76
 ts1        | /var/db/postgres0/ahq84
(4 строки)
```

```
uglyorangebird=> SELECT t.spcname, c.relname
FROM pg_class c
JOIN pg_tablespace t ON c.reltablespace = t.oid
WHERE c.relkind = 'r';
  spcname   |        relname
------------+-----------------------
 ts1        | event_logs_2025
 ts0        | event_logs_2026
 pg_global  | pg_authid
 pg_global  | pg_subscription
 pg_global  | pg_database
 pg_global  | pg_db_role_setting
 pg_global  | pg_tablespace
 pg_global  | pg_auth_members
 pg_global  | pg_shdepend
 pg_global  | pg_shdescription
 pg_global  | pg_replication_origin
 pg_global  | pg_shseclabel
 pg_global  | pg_parameter_acl
(13 строк)
```

Вывод

В ходе выполнения лабораторной работы был успешно создан и настроен кластер PostgreSQL с использованием выделенного узла.